`aligned_accessor`: An mdspan accessor expressing pointer over-alignment

Document #:	P2897R7
Date:	2024-11-22
Project:	Programming Language C++ LEWG

Reply-to:	Mark Hoemmen <mhoemmen@nvidia.com> Damien Lebrun-Grandie <lebrungrandt@ornl.gov> Nicolas Morales <nmmoral@sandia.gov> Christian Trott <crtrott@sandia.gov>

Contents

§ 1 Authors

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)
- Damien Lebrun-Grandie (lebrungrandt@ornl.gov) (Oak Ridge National Laboratory)
- Nicolas Morales (nmmoral@sandia.gov) (Sandia National Laboratories)
- Christian Trott (crtrott@sandia.gov) (Sandia National Laboratories)

§ 2 Revision history

- Revision 0 (pre-Varna) to be submitted 2023-05-19

- Revision 1 (pre-Kona) to be submitted 2023-10-15
  - Implement changes requested by LEWG review on 2023-10-10
    - Change `gcd` converting constructor Constraint to a Mandate
    - Add Example in the wording section that uses `is_sufficiently_aligned` to check the pointer over-alignment precondition
    - Add Example in the wording section that uses `aligned_alloc` to create an over-aligned allocation, to show that `aligned_accessor` exists as part of a system
    - Add an `explicit` constructor from default_accessor, so that users can type `aligned_mdspan y{x}` instead of `aligned_mdspan y{x.data_handle(), x.mapping()}`. Add an explanation in the design discussion section.
  - Implement other wording changes
    - Add to `aligned_accessor`'s Mandates that `byte_alignment >= alignof(ElementType)` is `true`. This prevents construction of an invalid `aligned_accessor` object.
  - Add more design discussion based on LEWG review on 2023-10-10
    - Explain why we do not include an `aligned_mdspan` alias
    - Explain `aligned_accessor` construction safety
- Revision 2 (post - St. Louis) to be submitted 2024-07-15
  - Implement required changes from LEWG review of R1 on 2024-06-28
    - Remove `constexpr` from `is_sufficiently_aligned`
  - Discuss optional suggestions from LEWG review of R1 on 2024-06-28
    - Add `explicit` converting constructor vs. named cast ("`naughty_cast`") discussion
    - Add `detectably_invalid` discussion
    - Ask LEWG to consider the alternative design that makes `is_sufficiently_aligned` a nonmember function in `<bit>` instead of a member function of `aligned_accessor`, while LWG review of R2 proceeds concurrently
  - P2389R2 was voted into the Working Draft at St. Louis, so replace use of `dextents` in examples with `dims`.
  - Add non-wording section explaining why `aligned_accessor` has no `explicit` constructor from less to more alignment

    - Add Compiler Explorer link with full implementation and demo

- Revision 3 (post - St. Louis) to be submitted 2024-07-15
    - Include updated feedback from David Sankel (see Acknowledgments) after his review of R2

- Revision 4 (post - St. Louis) to be submitted 2024-07-24
    - Make `is_sufficiently_aligned` a nonmember function instead of a static member function of `aligned_accessor`. R3 presented this only as an alternative. R4 makes this the actually proposed design.

- Revision 5 (post - St. Louis) to be submitted by 2024-08-15
    - Move `is_sufficiently_aligned` from `<bit>` to `<memory>`, due to feedback from LEWG mailing list review of R4
    - Give `is_sufficiently_aligned` a "*Throws*: Nothing" clause and add nonwording text explaining why

- Revision 6 to be submitted after LWG review
    - LWG reviewed the paper via virtual meeting 2024-10-25 and 2024-11-01. The second meeting did not have quorum, but attendees walked through the entire wording. LWG plans to see this paper again.
    - Change all template parameter names to be PascalCase, per Library convention (the only exceptions are `charT` and `traits`).
    - Swap order of template parameters of `is_sufficiently_aligned`.
    - Use `assume_aligned` in `offset` as well as in `access`.
    - Change `gcd` requirement in `aligned_accessor` converting constructor back from Mandate to Constraint. Add explanation with example in nonwording Section 5.9. Since both alignments are powers of two, just say "`OtherByteAlignment >= byte_alignment` is `true`."
    - Remove `access` Precondition, since it is implied by the Effects being equivalent to using `assume_aligned`.
    - Update Compiler Explorer implementation link.
    - Add alignment precondition to `aligned_accessor` class, and add nonwording section "Standard accessors already impose preconditions that propagate to `mdspan` construction" that explains the "class-wide" preconditions on data handles given to `default_accessor` and `aligned_accessor`.

- Make conversion operator to `default_accessor` `noexcept`, as is the converting constructor from greater to lesser alignment.
- Fix accessible range preconditions of `access` and `offset` to use index ranges instead of pointer ranges.
- Make conversion operator to `default_accessor` templated on the result's element type.
- Remove second Example (the long one), and move first example to right after the class overview.

- Revision 7 (during Wrocław) to be submitted 2024-11-22
  - Purely editorial change, requested by the editors: Mark all added sections in green text.
  - Purely editorial change: Make it clear that "Members [mdspan.accessor.aligned.members]" is a section to be added, and not just a section of the proposal.
  - Purely editorial change: Remove Editorial note ("Condition 5.2 is new as of version 6").
  - Purely editorial change: Fix paragraph numbers in [mdspan.accessor.aligned.members].

§ 3 Purpose of this paper

We propose adding `aligned_accessor` to the C++ Standard Library. This class template is an `mdspan` accessor policy that uses `assume_aligned` to decorate pointer access. We think it belongs in the Standard Library for two reasons. First, it would serve as a common vocabulary type for interfaces that take `mdspan` to declare their minimum alignment requirements. Second, it extends to `mdspan` accesses the optimizations that compilers can perform to pointers decorated with `assume_aligned`.

`aligned_accessor` is analogous to the various `atomic_accessor_*` templates proposed by P2689. Both that proposal and this one start with a Standard Library feature that operates on a "raw" pointer (`assume_aligned` or the various `atomic_ref*` templates), and then propose an `mdspan` accessor policy that straightforwardly wraps the lower-level feature.

We had originally written `aligned_accessor` as an example in P2642, which proposes "padded" mdspan layouts. We realized that `aligned_accessor` was more generally applicable and that standardization would help the padded layouts proposed by P2642 reach their maximum value.

§ 4 Key features

- `offset_policy` is `default_accessor`
- `data_handle_type` is `ElementType*`
- Permitted implicit conversions
  - from nonconst to const `ElementType`,
  - from more over-alignment to less over-alignment, and
  - from over-alignment to no over-alignment (`default_accessor`)
- `explicit` converting constructor from `default_accessor` lets users assert over-alignment
- New nonmember function `is_sufficiently_aligned` lets users check a pointer's alignment before using it with `aligned_accessor`

The `offset_policy` alias is `default_accessor<ElementType>`, because even if a pointer `p` is aligned, `p + i` might not be.

The `data_handle_type` alias is `ElementType*`. It needs no further adornment, because alignment is asserted at the point of access, namely in the `access` function. Some implementations might have an easier time optimizing if they also apply some implementation-specific attribute to `data_handle_type` itself. Examples of such attributes include `__declspec(align_value(byte_alignment))` and `__attribute__((align_value(byte_alignment)))`. However, these attributes should not apply to the result of `offset`, for the same reason that `offset_policy` is `default_accessor` and not `aligned_accessor`.

The converting constructor from `aligned_accessor` is analogous to `default_accessor`'s constructor, in that it exists to permit conversion from nonconst `element_type` to const `element_type`. It additionally permits implicit conversion from more over-alignment to less over-alignment – something that we expect users may need to do. For example, users may start with `aligned_accessor<float, 128>`, because their allocation function promises 128-byte alignment. However, they may then need to call a function that takes an `mdspan` with `aligned_accessor<float, 32>`, which declares the function's intent to use 8-wide SIMD of `float`.

The `explicit` converting constructor from `default_accessor` lets users assert that an `mdspan`'s pointer is over-aligned. This follows the idiom of existing `mdspan` layout mappings and accessors, where all conversions with preconditions are expressed as `explicit` constructors or conversion operators.

We do *not* provide an `explicit` conversion from an `aligned_accessor` with less alignment to an `aligned_accessor` with more alignment. As we explain below, we think

that if users need to do this conversion very often, then they likely have a design problem.

The `is_sufficiently_aligned` function checks whether a pointer has sufficient alignment to be used correctly with the class. This makes it easier for users to check preconditions, without needing to know how to cast a pointer to an integer of the correct size and signedness. As of R4 of this proposal, this is no longer a static member function of `aligned_accessor`. Instead, it is a nonmember function in the `<memory>` header.

§ 5 Design discussion

§ 5.1 The accessor is not nestable

We considered making `aligned_accessor` "wrap" any accessor type that meets the right requirements. For example, `aligned_accessor` could take the inner accessor as a template parameter, store an instance of it, and dispatch to its member functions. That would give users a way to apply multiple accessor "attributes" to their data handle, such as atomic access (see P2689) and over-alignment.

We decided against this approach for three reasons. First, we would have no way to validate that the user's accessor type has the correct behavior. We could check that their accessor's `data_handle_type` is a pointer type, but we could not check that their accessor's `access` function actually dereferences the pointer. For instance, `access` might instead interpret the pointer as a file handle or a key into a distributed data store.

Second, even if the inner accessor's `access` function actually did return the result of dereferencing the pointer, the outer `access` function might not be able to recover the effects of the inner `access` function, because `access` computes a `reference`, not a pointer. In order for `aligned_accessor`'s `access` function to get back that pointer, it would need to reach past the inner accessor's public interface. That would defeat the purpose of generic nesting.

Third, any way (not just this one) of nesting two generic accessors raises the question of order dependence. Even if it were possible to apply the effects of both the inner and outer accessors' `access` functions in sequence, it might be unpleasantly surprising to users if the effects depended on the order of nesting. A similar question came up in the "properties" proposal P0900, which we quote here.

> Practically speaking, it would be considered a best practice of a high-quality implementation to ensure that a property's implementation of `properties::element_type_t` (and other traits) are invariant with respect to ordering with other known properties (such as those in the standard library), but with this approach it would be impossible to make that guarantee formal, particularly with respect to other vendor-defined and user-defined properties unknown to the property implementer.

For these reasons, we have made `aligned_accessor` stand-alone, instead of having it modify another user-provided accessor.

§ 5.2 Explicit constructor from `default_accessor`

LEWG's 2023-10-10 review of R0 pointed out that in R0, `aligned_accessor` lacks an `explicit` constructor from `default_accessor`. Having that constructor would make it easier for users to create an aligned `mdspan` from an unaligned `mdspan`. Making it `explicit` would prevent implicit conversion. Thus, we have decided to add this `explicit` constructor in R1.

Without the `explicit` constructor, users have two options for turning a nonaligned `mdspan` into an aligned `mdspan`. First, as in the following example, users could "take apart" the input nonaligned `mdspan` and use the pieces to construct an aligned `mdspan`, whose type they name completely.

```
        void compute_with_aligned(
          std::mdspan<float, std::dims<2>, std::layout_left> matrix)
        {
          const std::size_t byte_alignment = 4 * alignof(float);
          using aligned_matrix_t = std::mdspan<float, std::dims<2>,
            std::layout_left, std::aligned_accessor<float,
byte_alignment>>;

          aligned_matrix_t aligned_matrix{matrix.data_handle(),
matrix.mapping()};
          // ... use aligned_matrix ...
        }
```

Second, as in the following example, users could construct an `aligned_accessor` explicitly and use constructor template argument deduction (CTAD) to construct the aligned `mdspan` from its pieces.

```
        void compute_with_aligned(
          std::mdspan<float, std::dims<2>, std::layout_left> matrix)
        {
          const std::size_t byte_alignment = 4 * alignof(float);

          std::mdspan aligned_matrix{matrix.data_handle(),
matrix.mapping(),
            std::aligned_accessor<float, byte_alignment>{}};
          // ... use aligned_matrix ...
        }
```

The first approach would likely be more common. This is because `mdspan` users commonly define their own type aliases for `mdspan`, with application-specific names that make code more self-documenting. The `aligned_matrix_t` definition above is an an example.

Adding an `explicit` constructor from `default_accessor` lets users get the same effect more concisely, without needing to "take apart" the input `mdspan`.

```
        void compute_with_aligned(std::mdspan<float, std::dims<2, int>,
std::layout_left> matrix)
        {
          const std::size_t byte_alignment = 4 * alignof(float);
          using aligned_mdspan = std::mdspan<float, std::dims<2, int>,
            std::layout_left, std::aligned_accessor<float,
byte_alignment>>;

          aligned_mdspan aligned_matrix{matrix};
          // ... use aligned_matrix ...
        }
```

The `explicit` constructor does not decrease safety, in the sense that users were always allowed to convert from an `mdspan` with `default_accessor` to an `mdspan` with `aligned_accessor`. Before, users could perform this conversion by typing the following.

```
        aligned_matrix_t aligned_matrix{matrix.data_handle(),
matrix.mapping()};
```

Now, users can do the same thing with fewer characters.

```
        aligned_matrix_t aligned_matrix{matrix};
```

§ 5.3 Why no explicit constructor from less to more alignment?

As explained in the previous section, `aligned_accessor` has an `explicit` converting constructor from `default_accessor` so that users can assert over-alignment. It also has an (implicit) converting constructor from another `aligned_accessor` with more alignment, to an `aligned_accessor` with less alignment. However, `aligned_accessor` does *not* have an `explicit` converting constructor from another `aligned_accessor` with *less* alignment, to an `aligned_accessor` with *more* alignment. Why not?

Consider the three typical use cases for `aligned_accessor`.

1. User knows an allocation's alignment at compile time.

2. User knows an allocation's alignment at run time, but not at compile time. For example, the value might depend on run-time detection of particular hardware features.

3. User doesn't know whether an allocation is over-aligned. They might need to ask some system at run time, or check the pointer value themselves, in order to decide whether to call code that expects a particular alignment.

In Case (1), users would normally declare the maximum alignment. They would want to preserve this information at compile time as much as possible, by keeping the `aligned_accessor` `mdspan` with maximum compile-time alignment for the entire scope of its use. Users would only want implicit conversions to less alignment or

`default_accessor` when calling functions whose parameter types encode these requirements.

Case (2) reduces to Case (3).

Case (3) reduces to Case (1). This works like any conversion from run-time type to compile-time type, with a fixed list of possible compile-time types (the alignments). As soon as a user's `mdspan` enters a scope where the alignment is known at compile time, the user would want to preserve that compile-time information and maximize the alignment for as large of a scope as possible.

None of these cases involve starting with more alignment, going to less (but still some) alignment, and then going back to more alignment again. Code that does that probably does not correctly use the types of function parameters to express its over-alignment requirements. It's like code that uses `dynamic_cast` a lot. Users can still convert from less or more alignment by creating the result's `aligned_accessor` manually. However, we don't want to encourage this pattern, so we don't offer an explicit conversion for it.

§ 5.4 We do not define an alias for aligned mdspan

In LEWG's 2023-10-10 review of R0, participants observed that this proposal's examples define an example-specific type alias for `mdspan` with `aligned_accessor`. They asked whether our proposal should include a *standard* alias `aligned_mdspan`. We do not *object* to such an alias, but we do not find it very useful, for the following reasons.

1. Users of `mdspan` commonly define their own type aliases whose names are meaningful for their applications.

2. It would not save much typing.

Examples may define aliases to make them more concise. One example in this proposal defines the following alias for an `mdspan` of `float` with alignment `byte_alignment`.

```
        template<size_t byte_alignment>
        using aligned_mdspan = std::mdspan<float, std::dims<1, int>,
          std::layout_right, std::aligned_accessor<float,
byte_alignment>>;
```

This lets the example use `aligned_mdspan<32>` and `aligned_mdspan<16>`.

The above alias is specific to a particular example. A *general* version of alias would look like this.

```
        template<class ElementType, class Extents, class Layout,
          size_t byte_alignment>
        using aligned_mdspan = std::mdspan<ElementType, Extents, Layout,
          std::aligned_accessor<ElementType, byte_alignment>>;
```

This alias would save *some* typing. However, mdspan "power users" rarely type out all the template arguments. First, they can rely on CTAD to create `mdspans`, and `auto` to return them. Second, users commonly already define their own aliases whose names have an application-specific meaning. They define these aliases *once* and use them throughout the application. For instance, users might define the following.

```
template<class ElementType>
using vector_t = std::mdspan<ElementType,
  std::dims<1>, std::layout_left>;
template<class ElementType>
using matrix_t = std::mdspan<ElementType,
  std::dims<2>, std::layout_left>;

template<class ElementType, size_t byte_alignment>
using aligned_vector_t = std::mdspan<ElementType,
  std::dims<1>, std::layout_left,
  std::aligned_accessor<ElementType, byte_alignment>>;
template<class ElementType, size_t byte_alignment>
using aligned_matrix_t = std::mdspan<ElementType,
  std::dims<2>, std::layout_left,
  std::aligned_accessor<ElementType, byte_alignment>>;
```

Such users may never type the characters "`mdspan`" again. For this reason, while we do not object to an `aligned_mdspan` alias, we do not find the proliferation of aliases particularly ergonomic.

§ 5.5 mdspan construction safety

LEWG's 2023-10-10 review of R0 expressed concern that `mdspan`'s constructor has no way to check `aligned_accessor`'s alignment requirements. Users can call `is_sufficiently_aligned` to check the pointer before constructing the `mdspan` with it. However, `mdspan`'s constructor generally has no way to check whether its accessor finds the caller's data handle acceptable.

This is true for any accessor type, not just for `aligned_accessor`. It is a design feature of `mdspan` that accessors can be stateless. Most of them have no state. Even if they have state, they generally do not store the data handle (as that would be redundant with the `mdspan`) and are thus generally not constructed with the data handle. As a result, an accessor might not see a data handle until `access` or `offset` is called. Both of those member functions are performance critical, so they cannot afford an extra branch on every call. Compare to `vector::operator[]`, which has preconditions but is not required to perform bounds checks. Using exceptions in the manner of `vector::at` could reduce performance and would also make `mdspan` unusable in a freestanding or no-exceptions context.

Note that `aligned_accessor` does not introduce *additional* preconditions beyond those of the existing C++ Standard Library feature `assume_aligned`. In the words of one LEWG reviewer, `aligned_accessor` is not any more "pointy" than `assume_aligned`; it just passes the point through without "blunting" it.

Before submitting R0 of this paper, we considered an approach specific to `aligned_accessor`, that would force the precondition back to `mdspan` construction time. This approach would wrap the pointer in a special data handle type with a constructor that takes a raw pointer, and has a precondition that the raw pointer has sufficient alignment. The constructor would be `explicit`, because it would have a precondition. The design would look something like this.

```
        template<class ElementType, std::size_t byte_alignment>
        class aligned_accessor {
        public:
          using element_type = ElementType;
          using reference = ElementType&;
          using offset_policy = stdex::default_accessor<ElementType>;

          class data_handle_type {
          public:
            constexpr data_handle_type() = default;

            // Checking the precondition can never be a compile-time
            // expression, so the constructor is not marked constexpr.
            explicit data_handle_type(element_type* the_data)
              : data_(the_data)
            { // Precondition: null, or sufficiently aligned.
              assert(data_ == nullptr ||
                is_sufficiently_aligned<byte_alignment>(data_));
            }

            // Conversion is implicit because it has no precondition.
            constexpr operator element_type* () const noexcept {
              return assume_aligned<byte_alignment>(data());
            }

          private:
            element_type* data_ = nullptr;
          };

          // ... the omitted parts of aligned_accessor would not change
...

          constexpr reference
            access(data_handle_type p, size_t i) const noexcept
          {
            return assume_aligned<byte_alignment>((element_type*)(p))[i];
          }

          constexpr typename offset_policy::data_handle_type
          offset(data_handle_type p, size_t i) const noexcept {
            return assume_aligned<byte_alignment>((element_type*)(p)) + i;
          }
        };
```

Users would have to construct the `mdspan` like this.

```
        element_type* raw_pointer = get_pointer_from_somewhere();
        using acc_type = aligned_accessor<element_type, byte_alignment>;
        mdspan x{acc_type::data_handle_type{raw_pointer}, mapping,
acc_type{}};
```

We rejected this approach in favor of `is_sufficiently_aligned` for the following reasons.

1. Wrapping the pointer in a custom data handle class would make every `access` or `offset` call need to reach through the data handle's interface, instead of just taking the raw pointer directly. The `access` function, and to some extent also `offset`, need to be as fast as possible. Their performance depends on compilers being able to optimize through function calls. The authors of `mdspan` carefully balanced generality with function call depth and other code complexity factors that may hinder compilers from optimizing. Performance of `aligned_accessor` matters as much or even more than performance of `default_accessor`, because `aligned_accessor` exists to communicate optimization potential.

2. The alignment precondition would still exist. Requiring the data handle type to throw an exception if the pointer is not sufficiently aligned would make `mdspan` unusable in a freestanding or no-exceptions context.

3. Users should not have to pay for unneeded checks. The two examples in the wording express the two most common cases. If users get a pointer from a function like `aligned_alloc`, then they already know its alignment, because they asked for it. If users are computing alignment at run time to dispatch to a more optimized code path, then they know alignment before dispatch. In both cases, users already know the alignment before constructing the `mdspan`.

4. The data handle is still a pointer, it's just a pointer with a constraint on its values. Users would reasonably expect to be able to use the result of `data_handle()` with existing interfaces that expect a raw pointer.

An LEWG poll on 2023-10-10, "[b]lock `aligned_accessor` progressing until we have a way of checking alignment requirements during `mdspan` construction," resulted in no consensus. Attendance was 14.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
0	**1**	**1**	**2**	**2**

LEWG expressed an (unpolled) interest that we explore `mdspan` safety in subsequent work after the fall 2023 Kona WG21 meeting. LEWG asked us to explore safety in a way that is not specific to `aligned_accessor`. Part of that exploration is in the section below "Generalize `is_sufficiently_aligned` for all accessors?". We plan further exploration of this topic elsewhere.

§ 5.6 `is_sufficiently_aligned` is not `constexpr`

LEWG reviewed R1 of this proposal at the June 2024 St. Louis WG21 meeting, and polled 1/10/0/0/1 (SF/F/N/A/SA) to remove `constexpr` from `is_sufficiently_aligned`. This is because it is not clear how to implement the function in a way that could ever be a constant expression. The straightforward cross-platform way to implement this would `bit_cast` the pointer to `uintptr_t`. However, `bit_cast` is not `constexpr` when converting from a pointer to an integer, per [bit.cast] 3. Any `reinterpret_cast` similarly could not be a core constant expression, per [expr.const] 5.15. One LEWG reviewer pointed out that some compilers have a built-in operation (e.g., Clang and GCC have `__builtin_bit_cast`) that might form a constant expression when `bit_cast` does not. On the other hand, the authors could not foresee a need for `is_sufficiently_aligned` to be `constexpr` and did not want to constrain implementations to use compiler-specific functionality.

§ 5.7 Generalize `is_sufficiently_aligned` for all accessors?

We proposed the `is_sufficiently_aligned` function so that users can check a pointer's alignment precondition before constructing an `aligned_accessor` `mdspan` with it. R4 of this paper changes `is_sufficiently_aligned` from a static member function of `aligned_accessor` to a nonmember function not in an `mdspan` header. C++ developers who do not use `mdspan` at all might still find `is_sufficiently_aligned` useful, for example to check the preconditions of `assume_aligned`.

Nevertheless, in the context of `mdspan` accessors, `is_sufficiently_aligned` is specific to `aligned_accessor`. No other `mdspan` accessors existing in or proposed for the Standard Library have an alignment precondition. Furthermore, `is_sufficiently_aligned` has a precondition that the pointer points to a valid element. Standard C++ offers no way for users to check that. More importantly for `mdspan` users, Standard C++ offers no way to check whether a pointer and a layout mapping's `required_span_size()` form a valid range.

For this reason, we do not propose here solving the general "is this data handle valid for an arbitrary given accessor?" question. That is, we do not propose adding a function to the accessor requirements that would tell if a given data handle and size pair is valid for that accessor. This section describes what such a check would look like if it existed.

§ 5.7.1 `detectably_invalid`: Generic validity check?

During the June 2024 St. Louis WG21 meeting, one LEWG reviewer (please see Acknowledgments below) pointed out that code that is generic on the accessor type currently has no way to check whether a given data handle is valid. Specifically, given a `size_t size` (e.g., the `required_span_size()` of a given layout mapping), there is no way to check whether [0, `size`) forms an accessible range (see [mdspan.accessor.general]

2) of a given data handle and accessor. The reviewer suggested adding a new member function

```
        bool detectably_invalid(data_handle_type handle, size_t size)
const noexcept;
```

to all `mdspan` accessors. This would return `true` if the implementation can show that [0, `size`) is *not* an accessible range for `handle` and the accessor, and `true` otherwise. The word "detectably" in the name would remind users that this is a "best effort" check. It might return `false` even if the handle is invalid or if [0, `size`) is not an accessible range. Also, it might return different values on different implementations, depending on their ability to check e.g., pointer range validity. The function would have the following design features.

— It must be a non-`static` member function, because in general, accessors may have state that determines validity of the data handle.

— It must be `const` because precondition-checking code should avoid observable side effects.

— It must be `noexcept` because precondition-checking code should not throw.

With such a function, users could write generic checked `mdspan` creation code like the following.

```
        template<class LayoutMapping, class Accessor>
        auto create_mdspan_with_check(
          typename Accessor::data_handle_type handle,
          LayoutMapping mapping,
          Accessor accessor)
        {
          if (accessor.detectably_invalid(handle,
mapping.required_span_size())) {
            throw std::out_of_range("Invalid data handle and/or size");
          }
          return mdspan{handle, mapping, accessor};
        }
```

§ 5.7.2 Arguments against and for `detectably_invalid`

We didn't include this feature in the original `mdspan` design because most data handle types have no way to say with full accuracy whether a handle and size are valid. We didn't want to give users the false impression that a validity check was doing anything meaningful. Standard C++ has no way to check a raw pointer `T*` and a size, though some implementations such as CHERI C++ ([Davis 2019] and [Watson 2020]) and run-time profiling and debugging systems such as Valgrind do have this feature. We designed `mdspan` accessors to be able to wrap libraries that implement a partitioned global address space (PGAS) programming model for accessing remote data over a network. (See P0009R18, Section 2.7, "Why custom accessors?".) Such libraries include the one-sided communication interface in MPI (the Message Passing Interface for distributed-memory

parallel programming) or NVSHMEM (NVIDIA's implementation of the SHMEM standard). Those libraries define their own data handle to represent remote data. For example, MPI uses an `MPI_Win` "window" object. NVSHMEM uses a C++ pointer to represent a "symmetric address" that points to an allocation from the "symmetric heap" (that is accessible to all participating parallel processes). Such libraries generally do not have validity checks for their handles.

On the other hand, a `detectably_invalid` function would let happen any checks that *could* happen. For instance, a hypothetical "GPU device memory accessor" (not proposed for the C++ Standard, but existing in projects like RAPIDS RAFT) might permit access to an allocation of GPU "device" memory from only GPU "device" code, not from ordinary "host" code. A common use case for GPU allocations is to allocate device memory in host code, then pass the pointer to device code for use there. Thus, it would be reasonable to create an `mdspan` in host code with that accessor. The accessor could use a CUDA run-time function like `cudaPointerGetAttributes` to check if the pointer points to valid GPU memory. Even `default_accessor` could have a simple check like this.

```
bool detectably_invalid(data_handle_type ptr, size_t size)
  const noexcept
{
  return ptr == nullptr && size != 0;
}
```

§ 5.7.3 Standard accessors already impose preconditions that propagate to `mdspan` construction

[mdspan.accessor.aligned.overview] 5 expresses class-wide preconditions on any data handle given to `aligned_accessor`'s `access` or `offset` member functions. The existing `default_accessor` has analogous preconditions in [mdspan.accessor.default.overview] 4. The reason we impose these preconditions on the entire accessor class, and not just `access` and `offset`, is that we intend for the preconditions to propagate to `mdspan` construction. That is, specializations of `mdspan` for `default_accessor` or `aligned_accessor` could, in theory, check the data handle given to `mdspan`'s constructor, by using the layout mapping's `required_span_size()` as the size of the range. We say "in theory" because C++ does not provide a Standard way to check whether a range is valid, but as we discussed above, some implementations do have that ability.

Implementations could thus give `default_accessor` and `aligned_accessor` their own "`detectably_invalid`" that `mdspan`'s constructor would use to check preconditions. Adding `detectably_invalid` to the accessor requirements would just extend this potential preconditions check to custom accessors.

§ 5.7.4 Users could work around the breaking change of adding `detectably_invalid` to accessor requirements

C++23 defines the generic interface of accessors through the accessor policy requirements **[mdspan.accessor.reqmts]**. Adding `detectably_invalid` to these requirements would be a breaking change to C++23. Thus, generic code that wanted to call this function would

need to fill in default behavior for both Standard accessors defined in C++23, and user-defined accessors that comply with the C++23 accessor requirements. The following `detectably_invalid` nonmember function (not proposed in this paper) shows one way users could do that. Please see Appendix A below for the full source code of a demonstration, along with a Compiler Explorer link. This demonstration shows that breaking backwards compatibility with C++23 is unnecessary, because users can straightforwardly work around the lack of a `detectably_invalid` member function in C++23 - compliant accessors. Not standardizing this nonmember function work-around would also give users the freedom to fill in different default behavior. For example, some users may prefer to consider every (data handle, size) pair invalid unless proven otherwise, as a way to force use of custom accessors that have the ability to make accurate checks.

```
        template<class Accessor>
        concept has_detectably_invalid = requires(Accessor acc) {
          typename Accessor::data_handle_type;
          { std::as_const(acc).detectably_invalid(
              std::declval<typename Accessor::data_handle_type>(),
              std::declval<std::size_t>()
            ) } noexcept -> std::same_as<bool>;
        };

        template<class Accessor>
        bool detectably_invalid(Accessor&& accessor,
          typename std::remove_cvref_t<Accessor>::data_handle_type handle,
          std::size_t size)
        {
          if constexpr
(has_detectably_invalid<std::remove_cvref_t<Accessor>>) {
            return std::as_const(accessor).detectably_invalid(handle,
size);
          }
          else {
            return false;
          }
        }
```

§ 5.7.5 `is_sufficiently_aligned` is still useful on its own

One could argue that if `aligned_accessor` had `detectably_invalid`, that would make `is_sufficiently_aligned` unnecessary. We disagree; we think `is_sufficiently_aligned` is useful by itself, whether or not `detectably_invalid` exists, for the following reasons.

1. Users will often want to check alignment separately from pointer range validity.

2. Checking alignment may be much less expensive than checking pointer range validity.

3. As of R4 of this paper, `is_sufficiently_aligned` is available without including an `mdspan` header, and thus is useful even to those who do not adopt `mdspan`.

Regarding (1), we think the most common use case for `aligned_accessor`'s `explicit` converting constructor from `default_accessor` would be explicit construction of an `mdspan` with `aligned_accessor` from an `mdspan` with `default_accessor`. The latter exists, so the user has already asserted that the range formed by its data handle and `required_span_size()` is valid. Thus, the only thing the user would need to check would be whether the data handle is sufficiently aligned.

The same LEWG reviewer who suggested `detectably_invalid` had originally thought it would make `is_sufficiently_aligned` unnecessary. However, after reviewing R2 of this paper, that reviewer changed their mind. They now agree with us that `is_sufficiently_aligned` is useful by itself. All their concerns would be addressed by making `is_sufficiently_aligned` a nonmember function, rather than a member function of `aligned_accessor`.

§ 5.7.6 Nonmember `is_sufficiently_aligned`

The reviewer responded to our argument above by suggesting that we remove `is_sufficiently_aligned` from `aligned_accessor` and make it a separate nonmember function. R4 of this paper implements this change.

§ 5.7.6.1 Mark it freestanding

We propose marking `is_sufficiently_aligned` freestanding. We know of no obstacles to this. Since `assume_aligned` is freestanding and since it would be reasonable to use `is_sufficiently_aligned` and `assume_aligned` together, it would make sense to mark `is_sufficiently_aligned` freestanding as well.

§ 5.7.6.2 Put it in `<memory>`

Into which header should this new function go? Since `is_sufficiently_aligned` does not depend on `mdspan`, it should not live in an `mdspan` header. It should be usable in any place that `assume_aligned` can be used. R4 proposed putting it in `<bit>`, because it is fundamentally a bit arithmetic operation. However, LEWG mailing list feedback expressed a strong preference for the function to go in `<memory>` instead. First, that would make it easier to use `is_sufficiently_aligned` and `assume_aligned` together. Second, "alignment is related to placement of the object in memory," as one LEWG mailing list reviewer pointed out. R5 thus proposes putting the function in `<memory>`.

§ 5.7.6.3 Throws: Nothing

R5 also adds a "*Throws*: Nothing" element to `is_sufficiently_aligned`. Users generally would not want `is_sufficiently_aligned` to throw, because it exists to check a precondition of `assume_aligned`.

Note that the function is *not* declared `noexcept`. This is because the function has a precondition, that its input `T* ptr` points to an object of a type similar to `T`. As we explained in the `detectably_invalid` discussion above, implementations do exist that

can check this precondition. In practice, the most common use cases for `is_sufficiently_aligned` are analogous to use of `dynamic_cast` for class hierarchies. Users start with a valid pointer with unknown alignment (analogous to a valid pointer to a base class `Base`), then assert or determine its alignment at run time (analogous to `dynamic_casting` the pointer to a subclass of `Base`, and checking if the result is null).

§ 5.7.7 Do accessors need to check anything else?

The only other thing an accessor's user might want to check besides a (data handle, size) pair would be converting construction from another type of accessor. All `mdspan` components – `extents`, layout mappings, and accessors – implement conversions with preconditions via `explicit` constructors. (For more detail, please see the section below, "Explicit conversions as the model for precondition-asserting conversions.") Accessors do *not* store their data handles, so the only reason to check whether converting construction is valid would be if the input or result accessor has separate run-time state. (Otherwise, the check could be a constraint or `static_assert`.) It's rare for an accessor to need run-time state, so we don't expect to need this feature in generic code. It would also be a separable addition from the feature of checking a data handle and size. Nevertheless, one could consider a design. We would favor just overloading `detectably_invalid` for accessors, as there would be no risk of ambiguity. Converting constructors only take one argument, so there would be no ambiguity between calling `detectably_invalid` with an accessor and calling it with a data handle and size.

§ 5.7.8 Naming the function

1. The function describes a property: "this (data handle, size) pair is not known to be invalid." It's an adjective (like "`valid`" or "`is_valid`"), not a verb (like "check" as in "`check_valid`").

2. The function does not promise perfect accuracy. In the common case, it says whether it can *detect* whether the handle and size are *not* valid. Whether they are *valid* might be harder to say.

3. As discussed above, users may also want to check converting constructors from other accessor types. However, there would be no risk of ambiguity between that and checking a data handle and size. Therefore, there's no need for the function's name to include the type of the thing being checked (e.g., "range").

4. Specifically, the function should not contain the word "pointer," because a data handle is not necessarily a pointer. Even if `data_handle_type` is a pointer type, a data handle might not necessarily be a pointer to the elements in the Standard C++ sense. For example, it might be some opaque handle that a library represents as a type alias of `void*`.

These points together suggest the name `detectably_invalid`.

§ 5.7.9 Conclusions

1. Adding `detectably_invalid` to the accessor requirements and existing Standard accessors in C++26 would be a breaking change to C++23. Nevertheless, even with this breaking change, users could still write code that fills in reasonable behavior for C++23 accessors.

2. Few C++ implementations offer a way to check validity of a pointer range. Thus, users would experience `detectably_invalid` as mostly not useful for the common case of `default_accessor` and other accessors that access a pointer range.

3. Item (1) reduces the urgency of adding `detectably_invalid` to C++26. Item (2) reduces its potential to improve the `mdspan` user experience in a practical way. Therefore, we do not suggest adding `detectably_invalid` to the accessor requirements in this proposal. However, we do not discourage further work in separate proposals.

4. R4 of this paper removes `is_sufficiently_aligned` from `aligned_accessor` and adds it to the Standard Library as a separate nonmember function. R5 puts it in the `<memory>` header.

§ 5.8 Explicit conversions as the model for precondition-asserting conversions

During the June 2024 St. Louis WG21 meeting, one LEWG reviewer asked about the `explicit` constructor from `default_accessor`. This constructor lets users assert that a pointer has sufficient alignment to be accessed by the `aligned_accessor`. The reviewer argued that this was an "unsafe" conversion, and wanted these "unsafe" conversions to be even more explicit than an `explicit` constructor: e.g., a new `*_cast` function template. We do not agree with this idea; this section explains why.

§ 5.8.1 Example: conversion to `aligned_accessor`

Suppose that some function that users can't change returns an `mdspan` of `float` with `default_accessor`, even though users know that the `mdspan` is over-aligned to `8 * sizeof(float)` bytes. The function's parameter(s) don't matter for this example.

```
        mdspan<float, dims<1>, layout_right, default_accessor<float>>
          overaligned_view(SomeParameters params);
```

Suppose also that users want to call some other function that they can't change. This function takes an `mdspan` of `float` with `aligned_accessor<float, 8>`. Its return type doesn't matter for this example.

```
        SomeReturnType use_overaligned_view(
          mdspan<float, dims<1>, layout_right, aligned_accessor<float,
8>>);
```

§ 5.8.2 Status quo

How do users call `use_overaligned_view` with the object returned from `overaligned_view`? The status quo offers two ways. Both of them rely on `aligned_accessor<float, 8>`'s `explicit` converting constructor from `default_accessor<float>`.

1. Use `mdspan`'s `explicit` converting constructor.

2. Construct the new `mdspan` explicitly from its data handle, layout mapping, and accessor. (This is the ideal use case for CTAD, as an `mdspan` is nothing more than its data handle, layout mapping, and accessor.)

Way (1) looks like this.

```
auto x = overaligned_view(params);
auto result = use_overaligned_view(
  mdspan<float, dims<1>, layout_right,
    aligned_accessor<float, 8>>(x)
);
```

Way (2) looks like this. Note use of CTAD.

```
auto x = overaligned_view(params);
auto result = use_overaligned_view(
  mdspan{x.data_handle(), x.mapping(),
    aligned_accessor<float, 8>>(x.accessor())}
);
```

Which way is less verbose depends on `mdspan`'s template arguments. Both ways, though, force the user to name the type `aligned_accessor<float, 8>` explicitly. Users know that they have pulled out a sharp knife from the toolbox. It's verbose, it's nondefault, and it's a class with a short definition. Users can go to the specification, see `assume_aligned`, and know they are dealing with a low-level function that has a precondition.

§ 5.8.3 `mdspan` uses explicit conversions to assert preconditions

The entire system of `mdspan` components was designed so that

— conversions with preconditions happen through `explicit` conversions (mostly converting constructors); while

— conversions without preconditions happen through implicit conversions.

Changing this would break backwards compatibility with C++23. For example, one can see this with converting constructors for

— `extents` (for conversions from run-time to compile-time extents, or conversions from wider to narrower index type): [mdspan.extents.cons]; and

— `layout_left::mapping`, and all the other layout mappings currently in the Standard that are not `layout_stride` or `layout_transpose` (for conversions from e.g.,

`layout_stride::mapping`, which assert that the strides are compatible): e.g., [mdspan.layout.left.cons].

This is consistent with C++ Standard Library class templates, in that construction asserts any preconditions. For example, if users construct a `string_view` or `span` from a pointer `ptr` and a size `size`, this asserts that the range [`ptr`, `ptr + size`) is accessible.

§ 5.8.4 Alternative: explicit cast function `naughty_cast`

Everything we have described above is the status quo. What did the one LEWG reviewer want to see? They wanted all conversions with preconditions to use a "cast" function with an easily searchable name, analogous to `static_cast`. As a placeholder, we'll call it "`naughty_cast`." For the above `use_overaligned_view` example, the `naughty_cast` analog of Way (2) would look like this.

```
auto x = overaligned_view(params);
auto result = use_overaligned_view(
  mdspan{x.data_handle(), x.mapping(),
    naughty_cast<aligned_accessor<float, 8>>>(x.accessor())}
);
```

One could imagine defining `naughty_cast` of `mdspan` by `naughty_cast` of its components. This would enable an analog of Way (1).

```
auto x = overaligned_view(params);
auto result = use_overaligned_view(naughty_cast<
  mdspan<float, dims<1>, layout_right,
    aligned_accessor<float, 8>>>(x)
);
```

Another argument for `naughty_cast` besides searchability is to make conversions with preconditions "loud," that is, easily seen in the code by human developers. However, the original Way (1) and Way (2) both are loud already in that they require a lot of extra code that spells out the result's accessor type explicitly. The status quo's difference in "volume" is implicit conversion

```
auto result = use_overaligned_view(x);
```

versus explicit construction.

```
auto result = use_overaligned_view(
  mdspan{x.data_handle(), x.mapping(),
    aligned_accessor<float, 8>(x)});
);
```

Adding `naughty_cast` to the latter doesn't make it much louder.

```
auto result = use_overaligned_view(
  mdspan{x.data_handle(), x.mapping(),
    naughty_cast<aligned_accessor<float, 8>>(x)});
);
```

There are other disadvantages to a `naughty_cast` design. The point of that design would be to remove or make non-`public` all the `explicit` constructors from `mdspan`'s components. That functionality would need to move somewhere. A typical implementation technique for a custom cast function is to rely on specializations of a struct with two template parameters, one for the input type and one for the output type of the cast. The `naughty_caster` struct example below shows how one could do that.

```
template<class Output, class Input>
struct naughty_caster {};

template<class Output, class Input>
Output naughty_cast(const Input& input) {
  return naughty_caster<Output, Input>::cast(input);
}

template<class OutputElementType, size_t ByteAlignment,
  class InputElementType>
  requires (is_convertible_v<InputElementType(*)[],
    OutputElementType(*)[]>)
struct naughty_caster {
  using output_type =
    aligned_accessor<OutputElementType, ByteAlignment>;
  using input_type = default_accessor<InputElementType>;

  static output_type cast(const input_type&) {
    return {};
  }
};
```

This technique takes a lot of effort and code, when by far the common case is that `cast` has a trivial body. For any accessors with state, it would almost certainly call for breaks of encapsulation, like making the `naughty_caster` specialization a `friend` of the input and/or output.

We emphasize that users are meant to write custom accessors. The intended typical author of a custom accessor is a performance expert who is not necessarily a C++ expert. It takes quite a bit of C++ experience to learn how to use encapsulation-breaking techniques safely; other approaches all just expose implementation details or defeat the "safety" that `naughty_cast` is supposed to introduce. Given that the main motivation of `naughty_cast` is safety, we shouldn't make it harder for users to write safe code.

More importantly, `naughty_cast` would obfuscate accessors. The architects of `mdspan` meant accessors to have to have a small number of "moving parts" and to define all those parts in a single place. Contrast `default_accessor` with the contiguous iterator requirements, for instance. The `naughty_cast` design would force custom accessors (and custom layouts) to define their different parts in different places, rather than all in one class. WG21 has moved away from this scattered design approach. For example, P2855R1 ("Member customization points for Senders and Receivers") changes P2300 (std::execution) to use member functions instead of `tag_invoke`-based customization points.

§ 5.8.5 Conclusion: retain `mdspan`'s current design

For all these reasons, we do not support replacing `mdspan`'s current "conversions with preconditions are explicit conversions" design with a cast function design.

§ 5.9 `gcd` requirement in converting constructor

LEWG had wanted the `gcd` requirement in `aligned_accessor`'s converting constructor to be a Mandate instead of a Constraint. LWG requested that we change it back, so that constructibility traits and overload resolution work as expected. LWG cites the following overload set as an example.

```
extern void compute(
  std::mdspan<float, std::dims<1>, std::layout_right,
    std::aligned_accessor<float, 16 * alignof(float)>> x);

extern void compute(
  std::mdspan<float, std::dims<1>, std::layout_right,
    std::aligned_accessor<float, 4 * alignof(float)>> x);
```

Suppose that the user has an 8x over-aligned mdspan `mdspan<float, dims<1>, aligned_accessor<float, 8 * alignof(float)>> x`, and calls `compute(x)`. With the Constraint design, the 4x overload would be called, which is the correct and expected behavior. With the Mandate design, the `compute(x)` call would be ambiguous.

§ 6 Implementation

We have tested an implementation of this proposal with the reference mdspan implementation. Appendix B below lists the source code of a full implementation.

§ 7 Example

```
template<size_t byte_alignment>
using aligned_mdspan = std::mdspan<
  float,
  std::dims<1, int>,
  std::layout_right,
  std::aligned_accessor<float, byte_alignment>>;

// Interfaces that require 32-byte alignment,
// because they want to do 8-wide SIMD of float.
extern void vectorized_axpy(
  aligned_mdspan<32> y, float alpha, aligned_mdspan<32> x);
extern float vectorized_norm(aligned_mdspan<32> y);

// Interfaces that require 16-byte alignment,
// because they want to do 4-wide SIMD of float.
extern void fill_x(aligned_mdspan<16> x);
extern void fill_y(aligned_mdspan<16> y);
```

```
        // Helper functions for over-aligned array allocations.

        template<class ElementType>
        struct delete_raw {
          void operator()(ElementType* p) const {
            std::free(p);
          }
        };

        template<class ElementType>
        using allocation =
          std::unique_ptr<ElementType[], delete_raw<ElementType>>;

        template<class ElementType, std::size_t byte_alignment>
        allocation<ElementType>
          allocate_raw(const std::size_t num_elements)
        {
          const std::size_t num_bytes = num_elements *
sizeof(ElementType);
          void* ptr = std::aligned_alloc(byte_alignment, num_bytes);
          return {ptr, delete_raw<ElementType>{}};
        }

        float user_function(size_t num_elements, float alpha)
        {
          // Code using the above two interfaces needs to allocate
          // to the max alignment.  Users could also query
          // aligned_accessor::byte_alignment for the various interfaces
          // and take the max.
          constexpr size_t max_byte_alignment = 32;
          auto x_alloc = allocate_raw<float, max_byte_alignment>
(num_elements);
          auto y_alloc = allocate_raw<float, max_byte_alignment>
(num_elements);

          aligned_mdspan<max_byte_alignment> x(x_alloc.get());
          aligned_mdspan<max_byte_alignment> y(y_alloc.get());

          // Two automatic conversions from 32-byte aligned to 16-byte
aligned
          fill_x(x);
          fill_y(y);

          // These interfaces use 32-byte alignment directly.
          vectorized_axpy(y, alpha, x);
          return vectorized_norm(y);
        }
```

§ 8 References

— Davis et al., “CheriABI: Enforcing Valid Pointer Provenance and Minimizing Pointer Privilege in the POSIX C Run-time Environment,” ASPLOS ’19, April 2019, pp. 379

- 393. Available online [last accessed 2024-07-05]: https://dl.acm.org/doi/10.1145/3297858.3304042

— Watson et al., “CHERI C/C++ Programming Guide,” Technical Report UCAM-CL-TR-947, University of Cambridge Computer Laboratory, June 2020. Available online [last accessed 2024-07-05]: https://doi.org/10.48456/tr-947

§ 9 Acknowledgments

— For `detectably_invalid`, credit (with permission) to David Sankel (Adobe), `dsankel@adobe.com`

§ 10 Wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording. The ﬠ character is used to denote a placeholder section number which the editor shall determine.

In **[version.syn]**, add

```
        #define __cpp_lib_aligned_accessor YYYYMML // also in <mdspan>
        #define __cpp_lib_is_sufficiently_aligned YYYYMML // also in
<memory>
```

Adjust the placeholder value `YYYYMML` as needed so as to denote this proposal’s date of adoption.

To the Header `<memory>` synopsis **[memory.syn]**, after the declaration of `assume_aligned` and before the declarations of functions in **[obj.lifetime]**, add the following.

```
        template<size_t Alignment, class T>
          bool is_sufficiently_aligned(T* ptr);
```

At the end of **[ptr.align]**, add the following.

```
        template<size_t Alignment, class T>
          bool is_sufficiently_aligned(T* ptr);
```

10 *Preconditions*: `p` points to an object `X` of a type similar (**[conv.qual]**) to `T`.

11 *Returns*: `true` if `X` has alignment at least `Alignment`, else `false`.

12 *Throws*: Nothing.

To the Header `<mdspan>` synopsis **[mdspan.syn]**, after `class default_accessor` and before `class mdspan`, add the following.

§ 10.1 Add `aligned_accessor` declaration to `<mdspan>` header synopsis

```
        // [mdspan.accessor.aligned], class template aligned_accessor
        template<class ElementType, size_t ByteAlignment>
          class aligned_accessor;
```

At the end of **[mdspan.accessor.default]** and before **[mdspan.mdspan]**, add the following.

§ 10.2 Add subsection 🟩 [mdspan.accessor.aligned] with the following

🟩 Class template `aligned_accessor` [mdspan.accessor.aligned]

🟩.1 Overview [mdspan.accessor.aligned.overview]

```
        template<class ElementType, size_t ByteAlignment>
        struct aligned_accessor {
          using offset_policy = default_accessor<ElementType>;
          using element_type = ElementType;
          using reference = ElementType&;
          using data_handle_type = ElementType*;

          static constexpr size_t byte_alignment = ByteAlignment;

          constexpr aligned_accessor() noexcept = default;

          template<class OtherElementType, size_t OtherByteAlignment>
            constexpr aligned_accessor(
              aligned_accessor<OtherElementType, OtherByteAlignment>)
noexcept;

          template<class OtherElementType>
            explicit constexpr aligned_accessor(
              default_accessor<OtherElementType>) noexcept;

          template<class OtherElementType>
          constexpr operator default_accessor<OtherElementType>() const
noexcept;

          constexpr reference access(data_handle_type p, size_t i) const
noexcept;

          constexpr typename offset_policy::data_handle_type
            offset(data_handle_type p, size_t i) const noexcept;
        };
```

1 *Mandates*:

(1.1) — `byte_alignment` is a power of two, and

(1.2) — `byte_alignment >= alignof(ElementType)` is `true`.

2 `aligned_accessor` meets the accessor policy requirements.

3 `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type.

4 Each specialization of `aligned_accessor` is a trivially copyable type that models `semiregular`.

5 $[0, n)$ is an accessible range for an object `p` of type `data_handle_type` and an object of type `aligned_accessor` if and only if

(5.1) — `[p, p + n)` is a valid range; and

(5.2) — if n is greater than zero, then `is_sufficiently_aligned<byte_alignment>(p)` is `true`.

[*Example:* The following function `compute` uses `is_sufficiently_aligned` to check whether a given `mdspan` with `default_accessor` has a data handle with sufficient alignment to be used with `aligned_accessor<float, 4 * sizeof(float)>`. If so, the function dispatches to a function `compute_using_fourfold_overalignment` that requires fourfold over-alignment of arrays, but can therefore use hardware-specific instructions, such as four-wide SIMD (Single Instruction Multiple Data) instructions. Otherwise, `compute` dispatches to a possibly less optimized function `compute_without_requiring_overalignment` that has no over-alignment requirement.

```
extern void
compute_using_fourfold_overalignment(
  std::mdspan<float, std::dims<1>, std::layout_right,
    std::aligned_accessor<float, 4 * alignof(float)>> x);

extern void
compute_without_requiring_overalignment(
  std::mdspan<float, std::dims<1>, std::layout_right> x);

void compute(std::mdspan<float, std::dims<1>> x)
{
  constexpr auto byte_alignment = 4 * sizeof(float);
  auto accessor =
    std::aligned_accessor<float, byte_alignment>{};
  auto x_handle = x.data_handle();

  if (std::is_sufficiently_aligned<byte_alignment>(x_handle)) {
    compute_using_fourfold_overalignment(
      std::mdspan{x_handle, x.mapping(), accessor});
  }
  else {
    compute_without_requiring_overalignment(x);
  }
}
```

–end example]

🔲.2 Members [mdspan.accessor.aligned.members]

```
        template<class OtherElementType, size_t OtherByteAlignment>
          constexpr aligned_accessor(
            aligned_accessor<OtherElementType, OtherByteAlignment>)
noexcept;
```

1 *Constraints*:

(1.1) — `is_convertible_v<OtherElementType(*)[], element_type(*)[]>` is `true`.

(1.2) — `OtherByteAlignment >= byte_alignment` is `true`.

2 *Effects*: None.

```
        template<class OtherElementType>
          explicit constexpr aligned_accessor(
            default_accessor<OtherElementType>) noexcept;
```

3 *Constraints*: `is_convertible_v<OtherElementType(*)[], element_type(*)[]>` is `true`.

4 *Effects*: None.

```
        constexpr reference
          access(data_handle_type p, size_t i) const noexcept;
```

5 *Preconditions*: [0, `i` + 1) is an accessible range for `p` and `*this`.

6 *Effects*: Equivalent to: `return assume_aligned<byte_alignment>(p)[i];`

```
        template<class OtherElementType>
          constexpr operator default_accessor<OtherElementType>() const
noexcept;
```

7 *Constraints*: `is_convertible_v<element_type(*)[], OtherElementType(*)[]>` is `true`.

8 *Effects*: Equivalent to: `return {};`

```
        constexpr typename offset_policy::data_handle_type
          offset(data_handle_type p, size_t i) const noexcept;
```

9 *Preconditions*: [0, `i` + 1) is an accessible range for `p` and `*this`.

10 *Effects*: Equivalent to: `return assume_aligned<byte_alignment>(p) + i;`

§ 11 Appendix A: `detectably_invalid` nonmember function example

This section is nonnormative. This is the full source code with tests for the `detectably_invalid` nonmember function example above. Please see this Compiler Explorer link for a test with five different compilers: GCC 14.1, Clang 18.1.0, MSVC v19.40 (VS17.10), and nvc++ 24.5.

```
        #include <cassert>
        #include <concepts>
        #include <cstdint>
        #include <iostream>
        #include <stdexcept>
        #include <type_traits>
        #include <utility>

        template<class Accessor>
        concept has_detectably_invalid = requires(Accessor acc) {
          typename Accessor::data_handle_type;
          { std::as_const(acc).detectably_invalid(
              std::declval<typename Accessor::data_handle_type>(),
              std::declval<std::size_t>()
            ) } noexcept -> std::convertible_to<bool>;
        };

        template<class Accessor>
        bool detectably_invalid(Accessor&& accessor,
          typename std::remove_cvref_t<Accessor>::data_handle_type handle,
          std::size_t size)
        {
          if constexpr
(has_detectably_invalid<std::remove_cvref_t<Accessor>>) {
            return std::as_const(accessor).detectably_invalid(handle,
size);
          }
          else {
            return false;
          }
        }

        struct A {
          using data_handle_type = float*;

          static bool detectably_invalid(data_handle_type ptr, std::size_t
size) noexcept {
            return ptr == nullptr && size != 0;
          }
        };

        struct B {
          using data_handle_type = float*;
        };

        struct C {
          using data_handle_type = float*;

          // This is nonconst, so it's not actually called.
          bool detectably_invalid(data_handle_type ptr, std::size_t size)
```

```
{
            throw std::runtime_error("C::detectably_invalid: uh oh");
          }
        };

        struct D {
          using data_handle_type = float*;

          // This is const but not noexcept, so it's not actually called.
          bool detectably_invalid(data_handle_type ptr, std::size_t size)
const {
            throw std::runtime_error("D::detectably_invalid: uh oh");
          }
        };


        int main()
        {
          float* ptr = nullptr;

          assert(not detectably_invalid(A{}, ptr, 0));
          assert(detectably_invalid(A{}, ptr, 1));

          A a{};
          assert(not detectably_invalid(a, ptr, 0));
          assert(detectably_invalid(a, ptr, 1));

          const A a_c{};
          assert(not detectably_invalid(a_c, ptr, 0));
          assert(detectably_invalid(a_c, ptr, 1));

          assert(not detectably_invalid(B{}, ptr, 0));
          assert(not detectably_invalid(B{}, ptr, 1));

          // B doesn't know how to check pointer validity.

          assert(not detectably_invalid(B{}, ptr, 0));
          assert(not detectably_invalid(B{}, ptr, 1));

          B b{};
          assert(not detectably_invalid(b, ptr, 0));
          assert(not detectably_invalid(b, ptr, 1));

          const B b_c{};
          assert(not detectably_invalid(b_c, ptr, 0));
          assert(not detectably_invalid(b_c, ptr, 1));

          // If users make detectably_invalid nonconst or not noexcept,
          // the nonmember function falls back to a default
implementation.

          try {
            assert(not detectably_invalid(C{}, ptr, 0));
            assert(not detectably_invalid(C{}, ptr, 1));
          }
```

```
          catch (const std::runtime_error& e) {
            std::cerr << "C{} threw runtime_error: " << e.what() << "\n";
          }

          try {
            const C c_c{};
            assert(not detectably_invalid(c_c, ptr, 0));
            assert(not detectably_invalid(c_c, ptr, 1));
          }
          catch (const std::runtime_error& e) {
            std::cerr << "const C threw runtime_error: " << e.what() <<
"\n";
          }

          try {
            C c{};
            assert(not detectably_invalid(c, ptr, 0));
            assert(not detectably_invalid(c, ptr, 1));
          }
          catch (const std::runtime_error& e) {
            std::cerr << "nonconst C threw runtime_error: " << e.what() <<
"\n";
          }

          try {
            assert(not detectably_invalid(D{}, ptr, 0));
            assert(not detectably_invalid(D{}, ptr, 1));
          }
          catch (const std::runtime_error& e) {
            std::cerr << "D{} threw runtime_error: " << e.what() << "\n";
          }

          try {
            const D d_c{};
            assert(not detectably_invalid(d_c, ptr, 0));
            assert(not detectably_invalid(d_c, ptr, 1));
          }
          catch (const std::runtime_error& e) {
            std::cerr << "const D threw runtime_error: " << e.what() <<
"\n";
          }

          try {
            D d{};
            assert(not detectably_invalid(d, ptr, 0));
            assert(not detectably_invalid(d, ptr, 1));
          }
          catch (const std::runtime_error& e) {
            std::cerr << "nonconst D threw runtime_error: " << e.what() <<
"\n";
          }

          std::cerr << "Made it to the end\n";
          return 0;
        }
```

§ 12 Appendix B: Implementation and demo

This Compiler Explorer link gives a full implementation of `aligned_accessor` and a demonstration. We show the full source code from that link here below.

```
        #include <https://raw.githubusercontent.com/kokkos/mdspan/single-
header/mdspan.hpp>
        #include <bit>
        #include <cassert>
        #include <cmath>
        #if defined(_MSC_VER)
        #  include <cstdlib> // MSVC's _aligned_malloc
        #endif
        #include <exception>
        #include <functional>
        #include <memory>
        #include <numeric>
        #include <type_traits>

        #define TEMPLATED_CONVERSION_TO_DEFAULT_ACCESSOR 1

        namespace stdex = std::experimental;

        // P2389 (voted into C++ at June 2024 STL plenary)
        namespace std {
        template<size_t Rank, class IndexType = size_t>
        using dims = dextents<IndexType, Rank>;

        template<size_t ByteAlignment, class ElementType>
        bool is_sufficiently_aligned(ElementType* p)
        {
          return bit_cast<uintptr_t>(p) % ByteAlignment == 0;
        }

        template<class ElementType, size_t ByteAlignment>
        class aligned_accessor {
        public:
          static constexpr size_t byte_alignment = ByteAlignment;

          static_assert(has_single_bit(byte_alignment),
            "byte_alignment must be a power of two.");
          static_assert(byte_alignment >= alignof(ElementType),
            "Insufficient byte alignment for ElementType");

          using offset_policy = stdex::default_accessor<ElementType>;
          using element_type = ElementType;
          using reference = ElementType&;
          using data_handle_type = ElementType*;

          constexpr aligned_accessor() noexcept = default;

          template<
```

```
            class OtherElementType,
            size_t OtherByteAlignment>
          requires(is_convertible_v<
            OtherElementType(*)[], element_type(*)[]> &&
            gcd(OtherByteAlignment, byte_alignment) == byte_alignment
          )
          constexpr aligned_accessor(
            aligned_accessor<OtherElementType, OtherByteAlignment>)
              noexcept
          {}

          template<class OtherElementType>
          requires(is_convertible_v<
            OtherElementType(*)[], element_type(*)[]>)
          constexpr explicit aligned_accessor(
            stdex::default_accessor<OtherElementType>) noexcept
          {}

        #if defined(TEMPLATED_CONVERSION_TO_DEFAULT_ACCESSOR)
          template<class OtherElementType>
            requires(is_convertible_v<
              element_type(*)[],
              OtherElementType(*)[]
            >)
          constexpr
            operator stdex::default_accessor<OtherElementType>() const
noexcept
        #else
          constexpr
            operator stdex::default_accessor<element_type>() const
noexcept
        #endif
          {
            return {};
          }

          constexpr reference
            access(data_handle_type p, size_t i) const noexcept
          {
            return assume_aligned<byte_alignment>(p)[i];
          }

          constexpr typename offset_policy::data_handle_type
          offset(data_handle_type p, size_t i) const noexcept {
            return assume_aligned<byte_alignment>(p) + i;
          }
        };

        } // namespace std

        namespace { // (anonymous)

        template<size_t byte_alignment>
        using aligned_mdspan =
          std::mdspan<float, std::dims<1, int>, std::layout_right,
```

```
            std::aligned_accessor<float, byte_alignment>>;

        // Interfaces that require 32-byte alignment,
        // because they want to do 8-wide SIMD of float.
        void
        vectorized_axpby(aligned_mdspan<32> y,
          float alpha, aligned_mdspan<32> x, float beta)
        {
          assert(x.extent(0) == y.extent(0));
          for (int k = 0; k < x.extent(0); ++k) {
            y[k] = beta * y[k] + alpha * x[k];
          }
        }

        // 1-norm of the vector y
        float vectorized_norm(aligned_mdspan<32> y)
        {
          float one_norm = 0.0f;
          for (int k = 0; k < y.extent(0); ++k) {
            one_norm += std::fabs(y[k]);
          }
          return one_norm;
        }

        // Interfaces that require 16-byte alignment,
        // because they want to do 4-wide SIMD of float.
        void fill_x(aligned_mdspan<16> x) {
          for (int k = 0; k < x.extent(0); ++k) {
            x[k] = static_cast<float>(k + 2);
          }
        }
        void fill_y(aligned_mdspan<16> y) {
          for (int k = 0; k < y.extent(0); ++k) {
            y[k] = static_cast<float>(k - 1);
          }
        }

        // Helper functions for making overaligned array allocations.

        template<class ElementType>
        struct delete_raw {
          void operator()(ElementType* p) const {
            std::free(p);
          }
        };

        template<class ElementType>
        using allocation =
          std::unique_ptr<ElementType[], delete_raw<ElementType>>;

        template<class ElementType, std::size_t byte_alignment>
        allocation<ElementType> allocate_raw(const std::size_t
num_elements)
        {
          const std::size_t num_bytes = num_elements *
```

```
sizeof(ElementType);
          float* ptr = reinterpret_cast<float*>(
        #if defined(_MSC_VER)
            _aligned_malloc(byte_alignment, num_bytes)
        #else
            std::aligned_alloc(byte_alignment, num_bytes)
        #endif
          );
          return {ptr, delete_raw<ElementType>{}};
        }

        float user_function(size_t num_elements, float alpha, float beta)
        {
          constexpr size_t max_byte_alignment = 32;
          auto x_alloc = allocate_raw<float, max_byte_alignment>
(num_elements);
          auto y_alloc = allocate_raw<float, max_byte_alignment>
(num_elements);

          aligned_mdspan<max_byte_alignment> x(x_alloc.get(),
num_elements);
          aligned_mdspan<max_byte_alignment> y(y_alloc.get(),
num_elements);

          // Implicit conversion from 32-byte aligned to 16-byte aligned
          fill_x(x);
          fill_y(y);

          // No conversion: interfaces expect 32-byte aligned and get it
          vectorized_axpby(y, alpha, x, beta);
          return vectorized_norm(y);
        }

        } // namespace (anonymous)

        namespace test_conversion_to_default_accessor {

        template<class ElementType>
        void
take_default_accessor_generic(stdex::default_accessor<ElementType>) {}

        template<class ElementType>
          requires(std::is_const_v<ElementType>)
        void
take_default_accessor_generic_const(stdex::default_accessor<ElementType>)
{}

        void take_default_accessor(stdex::default_accessor<float>) {}

        void take_default_accessor_const(stdex::default_accessor<const
float>) {}

        void test() {
          // Test new templated conversion operator to default_accessor.
          {
```

```
            std::aligned_accessor<float, 32> aligned_acc_f_nc;
            [[maybe_unused]] stdex::default_accessor<float> acc_f_nc{
aligned_acc_f_nc };
            [[maybe_unused]] stdex::default_accessor<float> acc_f_nc_2 =
aligned_acc_f_nc;
        #if defined(TEMPLATED_CONVERSION_TO_DEFAULT_ACCESSOR)
            [[maybe_unused]] stdex::default_accessor<const float> acc_f_c{
aligned_acc_f_nc };
            [[maybe_unused]] stdex::default_accessor<const float>
acc_f_c_2 = aligned_acc_f_nc;
        #endif

            // CTAD didn't work before anyway.
            //[[maybe_unused]] stdex::default_accessor acc_f{
aligned_acc_f_nc };

            take_default_accessor(aligned_acc_f_nc);
        #if defined(TEMPLATED_CONVERSION_TO_DEFAULT_ACCESSOR)
            take_default_accessor_const(aligned_acc_f_nc);
        #endif

            // Doesn't work either way.
            //take_default_accessor_generic(aligned_acc_f_nc);
          }
          {
            std::aligned_accessor<const float, 32> aligned_acc_f_c;
            [[maybe_unused]] stdex::default_accessor<const float> acc_f_c{
aligned_acc_f_c };
            [[maybe_unused]] stdex::default_accessor<const float>
acc_f_c_2 = aligned_acc_f_c;

            // CTAD didn't work before anyway.
            //[[maybe_unused]] stdex::default_accessor acc_f{
aligned_acc_f_c };

            take_default_accessor_const(aligned_acc_f_c);

            // Neither of these work either way.
            //take_default_accessor_generic(aligned_acc_f_c);
            //take_default_accessor_generic_const(aligned_acc_f_c);
          }
        }
        }

        int main(int argc, char* argv[])
        {
          float result = user_function(10, 1.0f, -1.0f);
          assert(result == 30.0f); // 3 + 3 + ... + 3 = 30
          test_conversion_to_default_accessor::test();

          return 0;
        }
```